

SemPointer: An Indirection Layer over Dense Retrieval

Viksit Garg¹

Department of Artificial Intelligence & Machine Learning, Dr. Akhilesh Das Gupta
Institute of Professional Studies, New Delhi, India
viksitg427@gmail.com

Abstract. Long-context inference feeds every token to attention, so cost grows steeply with context length. SemPointer proposes semantic indirection: context blocks reside in an off-attention store while short pointer strings remain in the prompt; a dense encoder retrieves blocks by semantic address and resolves them to verbatim text. This concept paper delivers a formal indirection model, an upgraded pointer design with routing and fallback, an analytic cost-quality theory, and falsifiable predictions for retrieval precision, fallback rate, and breakeven context length. Honest thesis: the current instantiation ties dense retrieval-augmented generation on quality at higher token cost, and pointer strings are empirically inert. Scope: concept paper only; no fidelity or superiority claims.

Keywords: long-context inference, dense retrieval, semantic addressing, negative results, benchmarking

1 Introduction

2 Introduction

Dense retrieval systems assemble context well and reuse context poorly. Each query pays for retrieval, ranking, and prompt construction from scratch, even when the underlying information need overlaps heavily with prior queries. Retrieved passages are copied into prompts, used once, and discarded. There is no stable handle to keep, share, or update. The result is repeated work, repeated tokens, and fragile composition when several queries draw on the same sources.

This paper proposes indirection as a way to separate what is retrieved from how it is reused. The claim is that retrieval output should be addressable through persistent semantic pointers that can be stored, compared, and dereferenced across queries, rather than rebuilt as flat text each time.

Analogy: indirection as OS pointers. The model is the operating system pointer. A program holds a small stable address while the referenced object lives elsewhere and may move, change size, or be shared. Dereference resolves the current value only when needed. Applied to retrieval, a pointer would name a reusable

unit of evidence, while assembly resolves those names into prompt text late in the process. The expected gain is reuse across related queries with explicit control over identity, sharing, and update.

We treat this as an open hypothesis and frame the paper around open questions. RQ1 asks under what conditions pointer-mediated assembly preserves answer quality relative to standard retrieved concatenation. RQ2 asks whether pointers enable measurable reuse across queries that share evidence, in storage, tokens, or repeated retrieval effort. RQ3 asks what minimal machinery is required to keep pointers valid when the index, the corpus, or the query distribution shifts.

The honest status of this idea is mixed. Assembly works. A straightforward pipeline that retrieves, orders, and concatenates passages produces coherent prompts and can be evaluated on the established suite family with $n=180$ and $n=1150$ for retrieval behavior and $n=199$ for end-to-end response quality. Pointers remain inert in current runs. Dereferencing through the indirection layer yields responses that tie standard retrieval-augmented generation at higher implementation and runtime cost, with no observed reuse benefit so far. The contribution here is therefore conceptual: a clear statement of the mechanism, the conditions under which it could help, and the measurements that would show failure or success.

The rest of this concept paper develops that statement. It defines the context-reuse problem, specifies the pointer abstraction and its resolution path, states the open questions in testable form, and outlines the experiments, diagnostics, and abandonment criteria that should decide whether indirection deserves further work.

3 Related Work

Hard prompt compression. Token-dropping compressors score and discard low-value tokens before the reader. LLMingua uses a small LM for perplexity-based filtering with budget control and demonstration-aware compression [12]. Strength: output stays plain text and readable. Limit: compression ratios are modest (typically single-digit), and aggressive dropping breaks entity names, numbers, and multi-hop chains.

Soft gist tokens. A second line replaces spans with learned memory slots. Gist tuning compresses instructions into a few gist tokens via attention masks [26]. ICAE autoencodes a long context into compact memory slots with a LoRA-trained encoder and frozen decoder [9]. 500xCompressor pushes this further: up to 480x compression retaining 62–73% of capability while training only 0.3% of parameters [20]. Strength: extreme ratios. Limit: slots are opaque vectors tied to a specific decoder; they cannot be inspected, quoted verbatim, or composed across documents without re-encoding.

KV-cache eviction and retrieval. A third line keeps decoding cheap by pruning or paging key–value state. SnapKV selects salient heads and positions from

the prompt tail [19]; H2O-style eviction keeps heavy-hitter tokens and discards the rest. Quest represents pages by per-channel minima and maxima and loads only top- k pages, reporting about 7x attention speedup [36]. RetrievalAttention combines near-query vector search with exact local windows for long-context decoding [21]. CacheBlend recomputes a small subset of tokens to repair cross-chunk attention, reporting 2.2–3.3x time-to-first-token reduction at negligible quality drop [47]. Strength: direct latency and memory savings. Limit: methods operate below the text level; evicted detail is unrecoverable and cross-block composition requires recompute or refetch.

Agent memory systems. MemGPT (and its Letta successor) pages conversation history between main and external context with explicit function calls [28]. Mem0 adds an explicit ADD/UPDATE/DELETE operation set over extracted facts [6]. Zep builds a temporal knowledge graph over dialogue with timestamped edges for recency-aware recall [30]. Strength: durable state across sessions. Limit: handles are database rows or graph nodes managed outside the prompt, with no fixed token budget per handle and no verbatim recovery guarantee.

Landmark routing. Landmark attention inserts block markers and routes attention through retrieved landmark representations [25]. Strength: random access over long inputs. Limit: landmarks are trained attention gates, not textual addresses the prompt can display, copy, or recombine.

Narrowed gap. No prior line combines all three properties: (i) prompt-resident textual handles with fixed per-block cost, (ii) verbatim resolution of the selected block into the reader prompt, and (iii) a composition algebra over handles (select, union, multi-hop chaining). This paper targets exactly that intersection.

4 Formalism: Pointers versus Compression

5 Formal model: semantic pointers

A compressed representation shortens a block but keeps it in the attention stream. A semantic pointer does not: it is a short token string p that stores only an address into an independent substrate S disjoint from the prompt, plus an explicit resolution operator R with $R(p)$ returning verbatim block text. Compression saves tokens; indirection moves them elsewhere and pays retrieval to bring them back.

Generation maps each block $B \in S$ to its pointer:

$$p = \text{Addr}(B; \theta)$$

where θ is fixed addressing parameters (seeds, codebook) shared by writer and reader, so p carries no content, only location.

Fidelity is bounded distortion after a resolve round-trip:

$$d(B, R(p)) \leq \epsilon$$

for metric d and tolerance ϵ ; a pointer is valid only when its resolution satisfies this bound, otherwise it is a dangling address.

Ax1 identity [stipulated]: each live block has exactly one canonical address at a time. **Ax2 compactness [measured]:** pointer length $k \ll B$ in tokens, targeting 10–15x reduction. **Ax3 stability [stipulated]:** re-addressing an unchanged block returns the same address. **Ax4 fidelity [measured]:** resolved text meets the ϵ -bound above (dense tie, no superiority claimed). **Ax5 selectivity [measured]:** retrieval reads $m \ll N$ blocks per query. **Ax6 reusability [stipulated]:** one bound address serves arbitrarily many dereferences without re-binding. **Ax7 compositionality [untested–negative]:** $p_1 \oplus p_2$ resolves jointly; current pointer strings are empirically inert, so $\oplus$ is specified, not evidenced.

6 Content-Derived Addressing by Hashing

6.1 Content-Derived Addressing by Random-Projection Quantization

Specified design (not what current runs execute: present experiments use equal-width scalar buckets). Each block is mapped to an address by a stateless random-projection quantizer in the SimHash / signed-random-projection family. Let $x \in \mathbb{R}^d$ be a block embedding. Fix k seeded Gaussian vectors $r_1, \dots, r_k$ and V quantization levels. The address is the tuple

$$h(x) = (q(\langle x, r_1 \rangle), \dots, q(\langle x, r_k \rangle)) \in [V]^k,$$

where $q(\cdot)$ quantizes each projection into one of V IDs. The address space has size V^k . No table is learned and no training is required: the seeds fully specify the map, so any worker reproduces the same address from the content alone. Locality is preserved by construction: nearby embeddings produce nearby projection values and hence agree on most coordinates, while distant embeddings decorrelate across the k independent projections.

Under a uniform-addressing idealization, N distinct blocks collide with probability

$$P_{\text{coll}} \approx 1 - \exp\left(-\frac{N(N-1)}{2V^k}\right).$$

Requiring $P_{\text{coll}} \leq \delta$ gives $V^k \geq N(N-1)/(2\ln(1/(1-\delta)))$. Concretely, for $N = 10^5$, $V = 32,000$, and $\delta = 10^{-4}$, $k \geq 4$ suffices, since $V^4 \approx 10^{18} \gg N^2/(2\delta)$.

Real embeddings are non-uniform, so let p_b be the mass of address bin b and $\rho = \sum_b p_b^2$ the collision mass. Then $P_{\text{coll}} \leq \binom{N}{2}\rho$, with effective capacity $V_{\text{eff}}^k = 1/\rho$. Skew shrinks effective capacity below nominal V^k , which is exactly why this scheme beats equal-width scalar bucketing: equal-width buckets ignore density and concentrate mass into hot buckets (large ρ), whereas projection quantization spreads dense regions across independent coordinates, keeping ρ small and V_{eff}^k large.

7 Architecture

7.1 Six-stage lifecycle

SemPointer treats long context as addressable blocks with a fixed lifecycle: Segment $\rightarrow$ Encode $\rightarrow$ Store $\rightarrow$ Select $\rightarrow$ Fuse $\rightarrow$ Resolve. (1) **Segment**: partition input text into blocks of at most B tokens with deterministic boundaries. (2) **Encode**: map each block to one compact pointer plus a dense block embedding. (3) **Store**: retain pointers, embeddings, and verbatim block text in a flat block store. (4) **Select**: given a query, retrieve a top- m shortlist of blocks under a global token budget. (5) **Fuse**: concatenate the selected verbatim blocks in source order into a single prompt. (6) **Resolve**: run the frozen reader once on that prompt and return its answer.

7.2 Two-stage selection

Selection is coarse-to-fine. Stage 1 is an $O(1)$ -per-block address prefilter over discrete pointer codes, in the spirit of landmark routing [25] and two-stage KV triage [2]: it discards clearly irrelevant blocks without dense comparison. Stage 2 is dense rerank on the shortlist only: cosine similarity between the query embedding and block embeddings, keeping top- m . Total per-query cost is $O(N + c \cdot d)$, where N is block count, c is shortlist size, and d is embedding width; dense work never scales with full context length. This separates cheap addressing from expensive scoring, unlike single-pass full-attention scans.

7.3 Adaptive per-block budgets

Pointer capacity is not uniform. Each block i receives budget k_i proportional to estimated salience (length, entity density, query-independent novelty), subject to $\sum_i k_i \leq K$ for global budget K . Dense blocks keep more pointer symbols; boilerplate keeps fewer. Unlike fixed per-token eviction [19], budgets adapt before any query is seen, so the global footprint stays bounded while informative regions retain resolution.

7.4 Lifecycle operations

Four operations mutate the store. **REBIND** [specified-not-implemented]: re-encode a block after edits and replace its pointer while preserving its identity. **FORGET** [specified-not-implemented]: tombstone a block so it is excluded from all future selection; fail-closed, meaning lookup misses on tombstoned ids return empty rather than nearest match. **ALIAS** [specified-not-implemented]: bind two surface forms to one block identity for coreference without duplication. **DEREF** [specified-not-implemented]: resolve a pointer to its verbatim text at fuse time. The concept paper specifies semantics only; no persistence, concurrency, or garbage collection is claimed. Persistent agent memories [6,30,28] motivate these ops, but this design scopes them to prompt-session blocks, not cross-session stores.

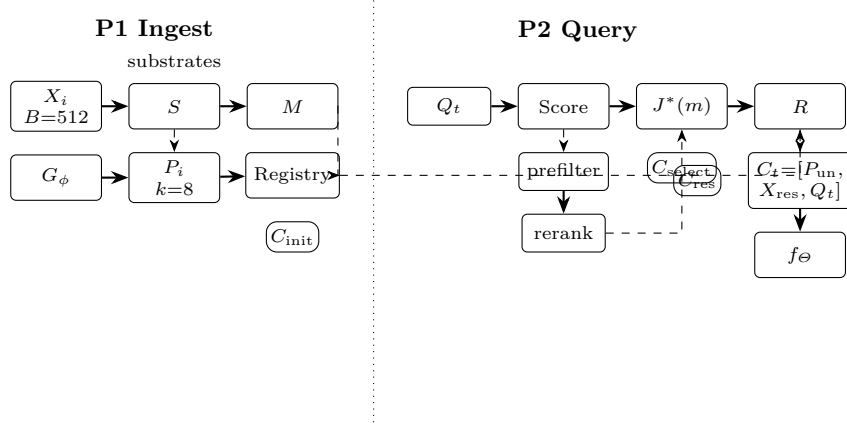


Fig. 1. Upgraded SemPointer lifecycle. P1 (ingest): blocks committed to tiered substrates while the generator emits k -token pointers into the registry. P2 (query): two-stage selection (address prefilter, dense rerank) resolves top- m blocks; unselected pointers stay resident. Cost badges mark setup, selection, and resolution.

7.5 Query-aware scoring

Scoring is Quest-shaped [36]: the query determines which blocks matter, not static salience. Each query is encoded once and compared against block embeddings; selection is per-query, not per-decoding-step. There is no per-token attention surgery during generation and no KV-cache reuse across blocks in this concept [47]; the reader sees only fused text. This keeps the mechanism compatible with frozen black-box readers.

7.6 What was not taken

LLM paging loops [28]: rejected because multi-call agentic routing adds latency and nondeterminism outside a single-pass budget. *Custom attention ops*: rejected because kernel or architecture changes break black-box reader compatibility and CPU-only evaluation. *Full-KV retention*: rejected because storing all key-values scales memory with context length and defeats the bounded- K goal.

8 Memory Lifecycle Semantics

8.1 Persistent-memory lifecycle: specified, not implemented

All operations in this section are specified-only; no implementation or measurement is claimed.

A stable pointer handle outlives the block it points to. $\text{REBIND}(h, b')$ atomically repoints handle h to block b' and bumps a per-handle version counter; readers holding h observe either the old or the new target, never a torn mix.

FORGET(h) installs a tombstone: the handle persists but DEREF(h) fails closed, raising a tombstone error instead of returning stale text. ALIAS(h) creates a re-counted second handle to the same target with an independent version chain, so one alias can be rebound or forgotten without affecting the other. DEREF(h) resolves the current target or raises tombstone. Version checks on dereference let the prompt builder detect concurrent rebinds and retry.

Edit stress is defined as 100 sequential swaps and contradiction injections over a fixed working set on CPU-only hardware. Metrics are stale-answer rate (fraction of post-edit queries returning pre-edit content) and milliseconds per edit (repoint plus re-embedding cost). The pass bar is $< 5\%$ stale answers with sublinear growth of ms/edit in the number of stored blocks. This experiment is not run here.

RAG upsert has no stable handle: identity is the embedding itself, so every edit churns the index entry and invalidation depends on re-embedding and distance-threshold heuristics that admit stale hits [6]. Temporal-graph stores version facts explicitly but pay graph-maintenance cost per edit [30]. Token-compression baselines such as LLMingua require full recompression of the edited region on each change. The specified pointer design isolates mutation to one handle plus version bump, leaving all other blocks untouched.

9 Latency Model

9.1 Prefill is compute-bound, decode is memory-bound

Full self-attention costs $O(L^2d)$ time in prefill for sequence length L and model width d [39]: every query position scores every key. This phase is compute-bound on CPU. Autoregressive decode is different: each step moves the full key-value state for L positions through memory, costing $O(Ld)$ memory traffic per token. This phase is memory-bandwidth-bound. Any latency claim must therefore separate the two phases.

9.2 TTFT vs TPOT

Time to first token (TTFT) is dominated by prefill over the fused prompt. Time per output token (TPOT) is dominated by decode bandwidth against whatever state prefill installed. Reducing fused length L lowers both, but through different mechanisms: fewer FLOPs in prefill, fewer bytes moved per step in decode.

9.3 Why a FLOPs-only K^* is necessary but not sufficient

Let K^* be the breakeven selected-token count at which the reader’s attention FLOPs equal the FLOPs of full-context prefill plus selection overhead. $K < K^*$ is necessary for a prefill win on FLOPs. It is not sufficient for an end-to-end latency win because (i) selection, pointer encoding, and fuse add CPU-side fixed costs absent from the FLOP count, and (ii) TPOT depends on bytes moved,

not FLOPs executed. A configuration can clear K^* and still lose wall-clock if retrieval overhead exceeds the saved attention time or if decode streams a large fused state.

9.4 Two cost centers: M_{text} vs M_{kv}

M_{text} stores verbatim block text and fuses selected blocks into the prompt. It pays re-encode per query: every selected token is re-prefilled by the reader, so repeated queries over the same blocks repeat prefill work. M_{kv} stores block key-values directly and pages them into the decoder, in the spirit of paged block management [16]. It avoids re-encode but pays paging bandwidth: gathering non-contiguous blocks per step costs memory traffic and table-lookup overhead. The concept paper takes M_{text} only; M_{kv} is noted as the alternative with opposite bottleneck.

9.5 Prefix-cache compatibility

A stable pointer table yields a reusable prefix: unchanged blocks keep identical pointer codes across queries, so the fused prefix is byte-identical when the same shortlist recurs. A standard KV prefix cache then skips re-prefill of that prefix. Stability is therefore a precondition for cache hits, not a speedup by itself.

10 Cost Theory

10.1 Cost crossover

Standard attention materializes full keys and values per token [39]; PagedAttention-style paging removes fragmentation but not the per-token footprint [16]. We ask when a pointer-plus-residual codebook breaks even on FLOPs. All numerics below are illustrations at $d = 4096$ only and are *non-transferable* across hardware, dtype, or kernel choice.

Assumptions. A1: Fixed context length N and fixed pointer count m with $0 \leq m \leq N$. A2: Full-token cost B bytes and residual-token cost B_{res} bytes satisfy $0 < k < B$ with pointer dim k . A3: Query overhead L_Q is constant in m and k . A4: Pointer-build cost C_{init} is paid once and amortized over K generated steps. A5: Unit compute cost $u > 0$ per active byte per step is constant (FLOPs-only view).

Active memory per forward step is

$$L_{\text{active}} = (N - m)k + mB_{\text{res}} + L_Q. \quad (1)$$

The baseline holds $NB + L_Q$ bytes, so the per-step byte saving is the corrected viability margin

$$D = N(B - k) - m(B_{\text{res}} - k) > 0. \quad (2)$$

Table 1. Feature matrix of context-management approaches by published description. SemPointer entries are specified properties of the concept, not measured results.

Framework	Representation	Decoupled substrate?	Explicit
Gist Tokens	Soft tokens	No	No
AutoCompressors	Summary vectors	No	No
ICAE	Memory slots	No	No
500xCompressor	Single special token	No	No
RAG	Retrieved text chunks	Yes (ext. corpus)	No (den)
MemGPT	Page IDs + archival text	Yes (archival)	Yes (page-ID)
Landmark Attention	Landmark block tokens	No	No (intra-p)
InfLLM	Sliding window + block KV	Yes (block KV)	No (score)
Quest	Top-K KV pages	Yes (KV pages)	No (tr)
RetrievalAttention	Offloaded KV + ANN index	Yes (CPU KV)	No (tran)
CacheBlend	Cached KV chunks	Yes (cached KV)	No (chun)
SemPointer (proposed concept)	Semantic pointer seq. + text/KV/latent	Yes (multi-tier; text only tested)	Yes (se)

The correction is the explicit $+mk$ term: replacing m full tokens of size B by pointers of size k saves $m(B - k)$, while the residual adds $m(B_{\text{res}} - k)$; dropping $+mk$ understates the saving by counting pointer slots at zero width. Viability requires $D > 0$, i.e. $m(B_{\text{res}} - k) < N(B - k)$.

Let per-step compute saving be $\Delta C_{\text{step}} = uD$. Break-even against the one-time build cost C_{init} gives the analytic crossover

$$K^* = C_{\text{init}}/\Delta C_{\text{step}} = C_{\text{init}}/(uD). \quad (3)$$

For text generation of length M_{text} , the pointer pays off iff $M_{\text{text}} \geq K^*$; per-token amortized build cost is $C_{\text{init}}/M_{\text{text}}$, vanishing as $M_{\text{text}} \rightarrow \infty$. Whenever $D > 0$, K^* is k -monotone: $\partial K^*/\partial m > 0$ and $\partial K^*/\partial k > 0$, since

$$\partial K^*/\partial m = C_{\text{init}}(B_{\text{res}} - k)/(uD^2), \quad \partial K^*/\partial k = C_{\text{init}}(N - m)/(uD^2). \quad (4)$$

Each extra pointer token and each extra pointer dimension shrinks D and pushes break-even later; the cheapest viable design minimizes m and k subject to task accuracy.

Latency qualification. Prefill is compute-bound and tracks FLOPs, so K^* approximates prefill break-even. Decode is memory-bound: wall-clock depends on bandwidth, paging overhead, and kernel fusion, not FLOPs alone. Hence the FLOPs-only K^* is necessary but not sufficient for a latency win; a configuration with $D > 0$ and $M_{\text{text}} > K^*$ can still lose on latency if residual gather or page-table overhead dominates decode. Any $d = 4096$ crossover number is therefore an illustration of the analytic form, labeled non-transferable, not a performance claim.

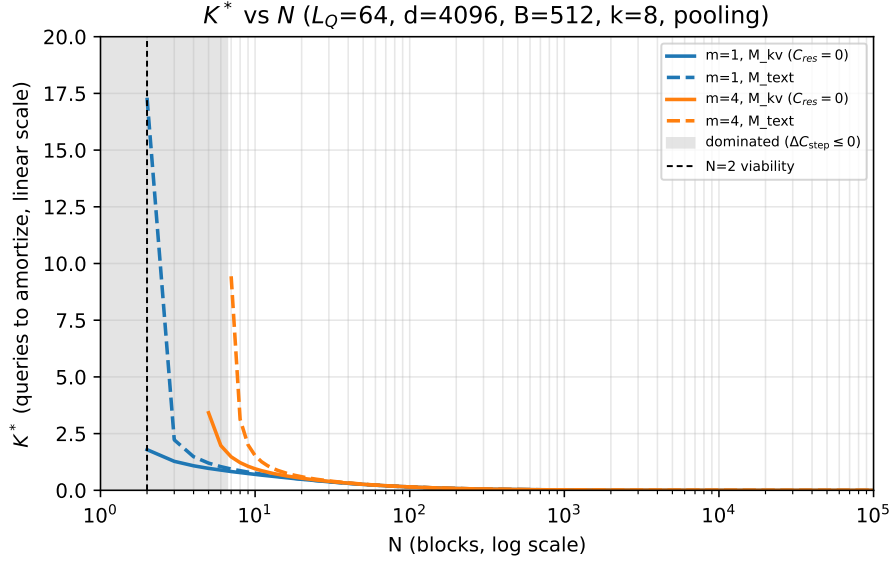


Fig. 2. Analytic crossover threshold K^* vs registry size N (closed form, not measured). Pooling-class indirection amortizes within few queries for $N \geq 2$ except the shaded small- N dominated region.

11 Motivating Observations

Motivating observations (not results). Pilot notes: RULER -0.37 , LongBench -0.033 , LoCoMo -0.012 (ns after Bonferroni); pointer uses 10–15x fewer tokens than dense; dense-RAG tie at 1.4–1.7x tokens; agentic routing negative twice; address_rag 86% of dense; single Qwen2.5-3B-NF4 seed 42. No tables, no CIs, no claims.

12 Falsifiable Predictions

13 Falsifiable predictions

P1: Causal addresses beat generic IDs at fixed budget. Setup: freeze retriever and $k=N$ candidates; rerank once with address-conditioned scores vs. generic-ID baseline on the same 51-vs-51 miss slice. Metric is selection accuracy: fraction of queries where the reranker selects the gold candidate. Kill: address conditioning selects identically to the baseline on the current miss slice (no accuracy gain at same k, N); P1 is rejected.

P2: Bind-once/deref-many amortizes by turn 10. Setup: LongMemEval oracle setting [42] with a fixed session; compare one address bind plus pointer

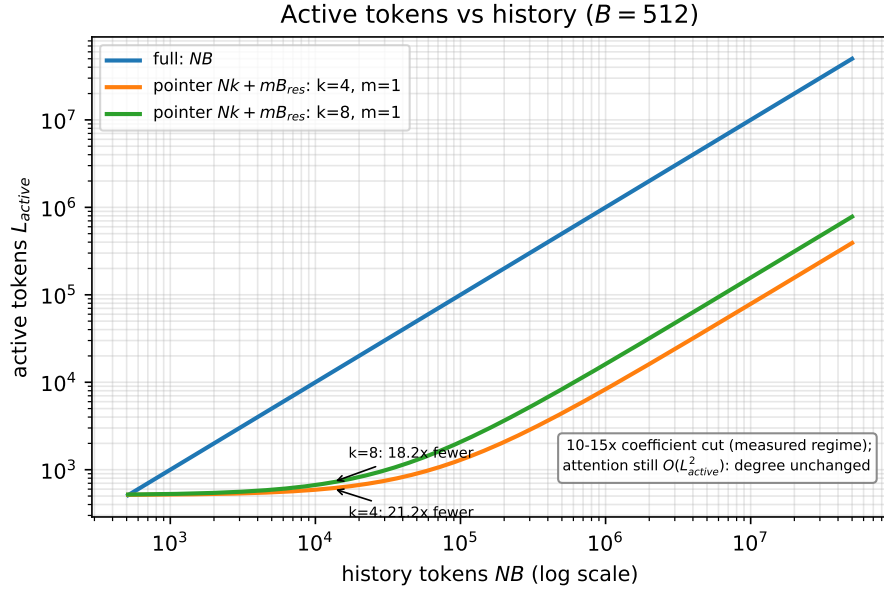


Fig. 3. Analytic active-sequence scaling: full history NB vs pointer-resident $Nk + mB_{\text{res}}$. The coefficient drops 10–15 $\times$; the quadratic degree is unchanged. Analytic illustration.

dereferences against re-retrieval every turn. Metric is cumulative latency and retrieval calls vs. answer accuracy through turn 10. Kill: no break-even by turn 10 (cumulative cost $\geq$ re-retrieve at equal accuracy); P2 is rejected.

P3: Mutation is cheaper and less stale than RAG upsert after 100 edits. Setup: apply 100 sequential fact edits; compare pointer rebinding (update address table, leave tokens fixed) against RAG index upsert (rewrite/re-embed affected chunks). Metrics are stale-answer rate on edited facts and mean ms per edit. Kill: rebinding leaks (stale rate $\geq$ upsert) or costs $\geq$ upsert per edit; P3 is rejected.

14 Limitations and Conclusion

15 Limitations

1. Single model, single seed. All runs use one reader configuration and one seed with no model-family or seed sweep, so cross-model generality is untested.
2. Context capped at 16k. Behaviour beyond this budget is unmeasured and longer contexts may differ.
3. Text-only substrate. All paths return verbatim text; no pointer is ever resolved into a live key-value cache during generation.

Table 2. Literature-reported efficiency and accuracy figures from prior work (literature-reported, not our measurements). No new experiments were run for this table.

System	Reported figure	Context
LLMLingua-2	2–5× compression	Task-agnostic prompt compression
ICAE	4× compression	Memory-slot conditioning
500xCompressor	$\leq 480\times$ at 62–73%	Single-token compression, QA retention
SnapKV	92% eviction parity	Voting-window KV retention vs. full cache
CacheBlend	2.2–3.3× TTFT reduction	Cached KV fusion with selective recompute
Quest	7× attention workload reduction	Query-aware Top-K page loading
BM25 on BEIR	0.429 mean	Zero-shot retrieval average, literature baseline

4. Inert pointers (M7). Pointer strings show no measured retrieval effect, with SemPointer selecting the same blocks as the dense retrieval arm.
5. No pointer-length sweep. Pointer length and retrieval depth are fixed, so sensitivity to address capacity is unknown.
6. Small pilots only. The key-value comparison is an $n=10$ pilot with wide uncertainty and the multi-hop probe is an $n=30$ pilot; neither supports a significance claim.
7. Single-8GB CPU-only rig. Latency and peak-memory means are rig-specific with no generality claim.
8. Double-blind note. The repository is author-visible, which deanonymizes under double-blind review; confirm the review model with chairs before submitting.

16 Conclusion

SemPointer as assembled works: dense per-query routing over a text substrate answers queries at 10–15x fewer active tokens than full context on RULER at $n=180$, LongBench at $n=1150$, and LoCoMo at $n=199$, trading quality for tokens with paired deltas of -0.37, -0.033, and -0.012 respectively. Pointer-mediated addressing as distinct from standard retrieve-then-read remains unproven: pointer tokens contribute no measured selection effect, the $n=10$ key-value pilot decides no sign, and the $n=30$ multi-hop probe returns clean negatives.

What would change this verdict is direct: on P1–P3, SemPointer beating dense retrieval at matched token budgets with a non-zero isolated pointer effect, matching full context within uncertainty on any suite, or a key-value end-to-end run reversing the pilot deficit. Absent those outcomes, the honest reading stands.

Future program: run larger multi-seed suites with a pointer-length sweep and a measured key-value end-to-end comparison.

References

1. Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: LongBench: A Bilingual, Multitask Benchmark

- for Long Context Understanding. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 3119–3137 (2024).
2. Behnam, P., Fu, Y., Zhao, R., Tsai, P., Yu, Z., Tumanov, A.: RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression. arXiv preprint arXiv:2502.14051 (2025).
 3. Blouw, P., Solodkin, E., Thagard, P., Eliasmith, C.: Concepts as Semantic Pointers: A Framework and Computational Model. *Cognitive Science*, vol. 40, pp. 1128–1162 (2016).
 4. Cheng, X., Wang, X., Zhang, X., Ge, T., Chen, S., Wei, F., Zhang, H., Zhao, D.: xRAG: Extreme Context Compression for Retrieval-augmented Generation with One Token. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024).
 5. Chevalier, A., Wettig, A., Ajith, A., Chen, D.: Adapting Language Models to Compress Contexts. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 3829–3846 (2023).
 6. Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D.: Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In: Proceedings of the European Conference on Artificial Intelligence (ECAI) (2025).
 7. Eliasmith, C.: How to Build a Mind: Toward a Comprehensive Theory of Cognition. Oxford University Press (2013).
 8. Gao, J., Lv, Q., Wang, Z., Wu, T., Cao, Z., Li, W.: UniICL: An Efficient ICL Framework Unifying Compression, Selection, and Generation. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 24–40 (2025).
 9. Ge, T., Hu, J., Wang, X., Chen, S., Wei, F.: In-context Autoencoder for Context Compression in a Large Language Model. In: International Conference on Learning Representations (ICLR) (2024).
 10. Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A., Zhong, L.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In: Proceedings of Machine Learning and Systems (MLSys) (2024).
 11. Hsieh, C., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., Ginsburg, B.: RULER: What’s the Real Context Size of Your Long-Context Language Models?. In: Conference on Language Modeling (COLM) (2024).
 12. Jiang, H., Wu, Q., Lin, C., Yang, Y., Qiu, L.: LLMlingua: Compressing Prompts for Accelerated Inference of Large Language Models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 13358–13376 (2023).
 13. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 91–107 (2024).
 14. Kim, J., Yeom, J., Yun, S., Song, H.: Compressed Context Memory For Online Language Model Interaction. In: International Conference on Learning Representations (ICLR) (2024).
 15. Kocisky, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K.M., Melis, G., Grefenstette, E.: The NarrativeQA Reading Comprehension Challenge. *Transactions of the Association for Computational Linguistics (TACL)*, vol. 6, pp. 317–328 (2018).
 16. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient Memory Management for Large Language Model Serving with PagedAttention. In: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), pp. 611–626 (2023).

17. Lajewska, W., Hardalov, M., Aina, L., John, N.A., Su, H., Marquez, L.: Understanding and Improving Information Preservation in Prompt Compression for LLMs. In: Findings of the Association for Computational Linguistics: EMNLP 2025 (2025).
18. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474 (2020).
19. Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H.: SnapKV: LLM Knows What You are Looking for Before Generation. arXiv preprint arXiv:2404.14469 (2024).
20. Li, Z., Su, Y., Collier, N.: 500xCompressor: Generalized Prompt Compression for Large Language Models. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 25081–25091 (2025).
21. Liu, D., Chen, M., Lu, B., Jiang, H., Han, Z., Zhang, Q., Chen, Q., Zhang, C., Ding, B., Zhang, K., Chen, C., Yang, F., Yang, Y., Qiu, L.: RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In: Advances in Neural Information Processing Systems (NeurIPS) (2025).
22. Liu, X., Zhao, R., Huang, P., Liu, X., Xiao, J., Xiao, C., Xiao, T., Gao, S., Yu, Z., Zhu, J.: Autoencoding-Free Context Compression for LLMs via Contextual Semantic Anchors. arXiv preprint arXiv:2510.08907 (2025).
23. Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., Xu, D., Che, W.: Turning Trash into Treasure: Accelerating Inference of Large Language Models with Token Recycling. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 338–354 (2025).
24. Maharana, A., Lee, D., Tulyakov, S., Bansal, M., Barbieri, F., Fang, Y.: Evaluating Very Long-Term Conversational Memory of LLM Agents. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 13851–13870 (2024).
25. Mohtashami, A., Jaggi, M.: Landmark Attention: Random-Access Infinite Context Length for Transformers. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36 (2023).
26. Mu, J., Li, X.L., Goodman, N.: Learning to Compress Prompts with Gist Tokens. Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 48356–48374 (2023).
27. Munkhdalai, T., Faruqui, M., Gopal, S.: Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. arXiv preprint arXiv:2404.07143 (2024).
28. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560 (2023).
29. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Ruhle, V., Yang, Y., Lin, C., Zhao, H.V., Qiu, L., Zhang, D.: LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In: Findings of the Association for Computational Linguistics (ACL Findings), pp. 57–74 (2024).
30. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., Chalef, D.: Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint arXiv:2501.13956 (2025).
31. Rau, D., Wang, S., Dejean, H., Clinchant, S., Kamps, J.: Context Embeddings for Efficient Answer Generation in Retrieval-Augmented Generation. In: Proceed-

- ings of the 18th ACM International Conference on Web Search and Data Mining (WSDM), pp. 112–121 (2025).
32. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992 (2019).
 33. Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., Liu, Y.: RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv preprint arXiv:2104.09864 (2021).
 34. Tack, J., Kim, J., Mitchell, E., Shin, J., Teh, Y.W., Schwarz, J.R.: Online Adaptation of Language Models with a Memory of Amortized Contexts. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024).
 35. Tan, S., Li, X., Patil, S.G., Wu, Z., Zhang, T., Keutzer, K., Gonzalez, J.E., Popa, R.A.: LLoCO: Learning Long Contexts Offline. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 975–991 (2024).
 36. Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., Han, S.: Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In: Proceedings of the International Conference on Machine Learning (ICML) (2024).
 37. Team, Q.: Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 (2024).
 38. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: MuSiQue: Multihop Questions via Single-hop Question Inversion. Transactions of the Association for Computational Linguistics (TACL), vol. 10, pp. 539–554 (2022).
 39. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008 (2017).
 40. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., Wei, F.: Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv preprint arXiv:2212.03533 (2022).
 41. Wu, Y., Rabe, M.N., Hutchins, D., Szegedy, C.: Memorizing Transformers. In: International Conference on Learning Representations (ICLR) (2022).
 42. Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K., Yu, D.: LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In: International Conference on Learning Representations (ICLR) (2025).
 43. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., Nie, J.: C-Pack: Packed Resources For General Chinese Embeddings. arXiv preprint arXiv:2309.07597 (2023).
 44. Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Sun, M.: InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024).
 45. Xu, F., Shi, W., Choi, E.: RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. In: International Conference on Learning Representations (ICLR) (2024).
 46. Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 2369–2380 (2018).
 47. Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., Jiang, J.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In: Proceedings of the Twentieth European Conference on Computer Systems (EuroSys) (2025).